

06 · 工具的定义、注册、选择、校验与执行

从 ToolDefinition/AgentTool 到参数校验、并行执行、ToolResult 回灌上下文

分析版本: 027a5847901b5dde30270abaa1041046cd2b4b55

核心结论

工具链分两层：Coding Agent 用 ToolDefinition 承载产品级描述/渲染/扩展上下文，再包装成 Agent Core 的 AgentTool；Agent Loop 根据 assistant.toolCall.name 查找工具，先 prepareArguments，再 validateToolArguments，经过 beforeToolCall，执行 execute，经过 afterToolCall，最后构造 ToolResultMessage 并追加上下文触发下一次模型调用。

1. 为什么有 ToolDefinition 和 AgentTool 两种形态

Agent Core 只需要执行工具所需的最小接口：name、description、parameters、prepareArguments、executionMode、execute。Coding Agent 还需要 prompt snippet、UI renderer、ExtensionContext/sourceInfo 等产品能力，因此内部 registry 采用 definition-first，再通过 wrapToolDefinition 转成 AgentTool 给 core。

2. 内置工具从哪里来

createAgentSession 默认激活 read、bash、edit、write；grep/find/ls 等也在 coding-agent 的 tools 目录定义，可通过 --tools/--exclude-tools/settings/extension 动态选择。AgentSession 建立完整 definition registry 后，再计算 active tool set 并写入 agent.state.tools。

3. 模型为什么能“看到”工具

Agent.state.tools 进入 AgentContext.tools；streamAssistantResponse 构造 pi-ai Context 时原样提供 tools。Provider adapter 再把统一 Tool 的 name/description/TypeBox schema 转成供应商工具定义。因此“注册工具”和“暴露给本轮模型”是不同动作：注册进 registry 不代表一定 active。

4. toolCall 选择真实实现

模型返回 ToolCall{name,id,arguments} 后，executeToolCalls 用 name 在 currentContext.tools 中查找。找不到时不会抛出未捕获异常，而是立即生成 error ToolResult，模型可在下一轮看到“tool not found”。这是 Agent 协议层的可恢复错误。

5. 参数处理的严格顺序

```
toolCall.arguments
-> tool.prepareArguments?
-> validateToolArguments(tool, preparedToolCall)
-> beforeToolCall(ctx)
    -> block? => error result
-> tool.execute(validatedArgs)
```

prepareArguments 允许工具在 schema 校验前做兼容/标准化；真正安全边界是 validateToolArguments。beforeToolCall 接收到的是已验证 args，因此权限扩展可以基于结构化参数判断，而不必解析未经验证的原始 JSON。

6. 执行期间的流式进度

execute 接收 onUpdate callback，工具可发送 partialResult；Agent Loop 把它转为 tool_execution_update。acceptingUpdates 标志在工具 settle 后立即关闭，并等待已经排队的 update event 完成。测试专门保护“late update 不应在结束后污染事件流”。

7. beforeToolCall 与 afterToolCall 的意义

Hook	时机	能力	典型用途
beforeToolCall	schema 校验后、execute 前	block/reason	危险命令审批、路径保护、权限策略
afterToolCall	execute 后、end event 前	替换 content/details/isError/usage/ terminate	脱敏、结果规范化、统计、提前停止

afterToolCall 是字段级覆盖，不做深合并。hook 自身异常会被转成 error tool result，保持 Agent 生命周期可继续。

8. 顺序与并行执行

默认 toolExecution=parallel。所有工具先按模型顺序做 preparation；允许的工具随后并发 execute。tool_execution_end 按完成时间发出，但最终 ToolResultMessage 按 assistant 的 source order 发出。若全局设 sequential，或任一工具 executionMode=sequential，则整批顺序执行。

9. 为什么 stopReason=length 时禁止执行任何工具

流式 tool arguments 可能在输出 token 限制处被截断。JSON salvage 可能把半截内容修成“可解析且恰好符合 schema”的对象，这会造成危险的静默错误。源码因此把 length + toolCall 当成不可信输入，整批生成错误结果，要求模型重发。

10. ToolResult 如何回到模型

finalized outcome 被包装为 role=toolResult, 含 toolCallId、toolName、content、details、usage、isError、timestamp。Agent Loop 先发 message_start/end, 再把 result 追加 currentContext.messages。下一次 provider request 的 convertToLlm 会保留 toolResult, Provider adapter 再转成 function_call_output / tool_result 等原生格式。

11. terminate 的精确语义

单个 result.terminate=true 不会自动终止整批; 只有本批每个 finalized result 都 terminate=true, batch.terminate 才为 true。这一点在并行执行中特别重要, 可避免一个工具单方面使其他工具的结果失去后续模型解释机会。

12. 修改工具时必须保持的不变量

- tool name 在注册、Provider schema、toolCall、toolResult 中必须稳定一致。
- 参数必须在 execute 前通过 TypeBox schema; 不要把校验责任移到 shell/文件系统之后。
- toolResultId/toolCallId 配对必须保留; Compaction 也不能把 result 与 call 切开。
- 工具取消应尊重 AbortSignal, 且不会在 settle 后继续发送 update。
- 并行文件写工具若存在共享资源冲突, 应声明 sequential 或使用额外 mutation queue。

动态验证方案

实验 1 : 未知工具

假设: 找不到工具应成为可见 error result 而非崩溃

操作: Mock assistant 返回 name=missing; 记录事件与下一轮 context。

预期: 出现 tool_execution_start/end + toolResult(isError=true), Agent 可继续。

若不一致: 若 throw 泄漏, 检查是否绕过 Agent Loop。

实验 2 : 参数校验

假设: execute 只接收 schema 验证后的参数

操作: 工具 schema 要求 number, 模型返回 string; execute 中设置计数器。

预期: execute 不被调用, 结果为错误 toolResult。

若不一致: 若执行了, 检查 prepareArguments 是否主动转换了类型。

实验 3 : 权限拦截

假设: beforeToolCall 能在 execute 前阻断

操作: 扩展对 bash rm 命令返回 block=true。

预期: 工具不执行, 模型收到 block reason。

若不一致: 若命令已运行, hook 安装顺序/ExtensionRunner 绑定有问题。

源码证据索引

文件	关键符号/用途	本报告结论
packages/agent/src/types.ts	AgentTool / ToolExecutionMode / before/after contexts	Core 工具契约
packages/agent/src/agent-loop.ts	prepareToolCall / validateToolArguments / execute* / createToolResultMessage	完整执行闭环
packages/coding-agent/src/core/tools/tool-definition-wrapper.ts	wrapToolDefinition	产品定义 -> Core AgentTool
packages/coding-agent/src/core/tools/*	read/bash/edit/write/grep/find/ls	内置工具实现
packages/coding-agent/src/core/agent-session.ts	tool registry / active tools / hooks	注册与选择
packages/agent/test/agent.test.ts	late update / parallel behavior 等	并发与事件契约测试

证据状态：除非单独标注，本报告中的行为结论均来自上述固定 commit 的静态源码与仓库测试/文档交叉验证；未实际启动 Pi、未抓取真实网络请求，因此“运行结果已验证”项为空。

推荐阅读顺序

优先级	文件	阅读目的	精读
1	packages/agent/src/types.ts	先理解 AgentTool 接口	是
2	packages/agent/src/agent-loop.ts	从 prepareToolCall 连读到 createToolResultMessage	是
3	packages/coding-agent/src/core/tools/tool-definition-wrapper.ts	理解两层工具模型	是
4	packages/coding-agent/src/core/tools/read.ts	选一个读工具看实现	否
5	packages/coding-agent/src/core/tools/edit.ts	再看一个有副作用的工具	是